Distributed Systems' Project
Ant Colony Optimization Problem

Daniele Ferroli (mat. 137357)
Luca Gemolotto (mat. 143865)
DMIF, University of Udine, Italy

Version 0.2, August 12, 2023

Abstract

The aim of this document is to describe how to write a report for the project-exam for the course Distributed Systems, at the University of Udine. It is also a guideline for the development of the project itself.

This document is very brief and succint, and it is by no means comprehensive of all the informations that should be given about a software project. You are invited to supplement these guidelines as needed to best describe your work.

Chapter 1

Introduzione

In this chapter you describe the main problem, and an idea of the solution. It is not necessary to be very detailed or formal, but it is important to explain which are the main aims and issues from the point of view of Distributed Systems:

- A description of the application.
- The overall structure of the implementation: how resources are deployed, which are the players, the rôles.
- The distributed system features (and the transparencies) and algorithms you intend to implement.
- Your plan for testing the system.
- A schedule for how you plan to carry our your design and implementation

1.1 Descrizione del problema

Il problema consiste nel fornire in tempo reale a un agente un approssimazione del cammino migliore per i suoi clienti. La topologia dei clienti viene fornita dall'utente in modo tale da poterla aggiornare in base alle sue necessità. A tal fine viene proposta una soluzione basata sull'algoritmo di "Ant Colony Optimization" (**ACO**).

Illustriamo a seguire la nostra proposta:

- l'utente può interfacciarsi al sistema per inserire i clienti e i costi per spostarsi tra quest'ultimi;
- il sistema crea un grafo e computa un'approssimazione del percorso migliore;
- tramite interfaccia grafica viene visualizzato il grafo e il percorso;
- l'utente può fornire nuovi dati sui clienti e/o i costi;
- il sistema si aggiorna mostrando il nuovo percorso all'agente.

Il sistema si presuppone di essere in continua esecuzione e che l'approsimazione del percorso migliore sia adeguata.

Infine, la soluzione proposta è applicabile anche ad altri contesti, con pochi adattamenti necessari, se non nessuno. Uno di questi contesti potrebbe essere un sistema di veicoli autonomi, in cui questi ultimi si muovono all'interno di un'area, comunicando informazioni riguardanti lo stato dei percorsi, dove si vogliono trovare i percorsi migliori.

1.2 Struttura dell'applicazione

L'applicazione è composta dai seguenti elementi:

- il sistema ACO distribuito, modulo logico per computare l'approssimazione del TSP;
- un'interfaccia grafica, per poter visualizzare il grafo e il cammino calcolato;
- un modulo intermedio tra sistema ACO, interfaccia grafica ed interazioni dell'utente (API);
- un agente (o multipli) che utilizza l'interfaccia grafica;
- un utente (o un sistema) che interagisce con il modulo intermedio, per aggiungere o rimuovere clienti e modificare informazioni sui percorsi.

Da questo punto in poi, con *agente* si intenderà l'ente che usa l'interfaccia grafica, mentre con *utente* si intende solamente chi utilizza le API del sistema.

1.2.1 Struttura distribuita

Il sistema ACO è un sistema distribuito peer-to-peer, i cui nodi non necessariamente hanno una conoscenza completa della topologia. All'interno del sistema possono essere presenti più nodi presenti su una stessa macchina, mentre quest'ultime comunicheranno tra di loro passando per i nodi.

I nodi, idempotenti, possono essere di tre tipi:

- Nodo agente: riceve informazioni sul grafo;
- Nodo utente: aggiorna il grafo dei clienti;
- Nodo ACO: gestisce i nodi del grafo dei clienti e partecipa alla computazione dell'algoritmo.

La comunicazione tra nodi è asincrona e accoppiata rispetto sia al tempo che allo spazio. Questo perché un nodo invia un messaggio a un suo vicino (specifico), il quale deve esistere nello stesso momento, senza però dover aspettare una risposta.

La topologia del sistema è rappresentata da un grafo pesato non orientato composto da clienti in cui gli archi sono le effettive distanze più eventuali perturbazioni, come ad esempio lo stato del traffico. I nodi del sistema ACO gestiscono un sottoinsieme dei clienti presenti.

1.2.2 Algoritmi e CAP theorem

Considerando il CAP theorem, si è scelto di assicurare disponibilità e tolleranza rispetto alle partizioni. Questo perché è necessario che il sistema sia sempre responsivo e possa funzionare anche se un nodo è oggetto di malfunzionamenti. Per quanto concerne la consistenza, nel caso di un'azienda di dimensioni ridotte, sarebbe preferibile rispetto alla disponibilità, poiché il grafo dei clienti non riceverebbe frequenti aggiornamenti. Nel nostro caso, invece, supponiamo di lavorare per un'azienda sufficientemente grande, con una discreta dinamicità dei clienti. Risulta quindi più importante poter aggiornare il grafo velocemente rispetto al fatto che l'intero sistema abbia le informazioni più recenti.

ECHO algorithm per mandare dati formiche fine round a tutti i nodi. No lista di nodi conosciuti, formica non può venire mandata in nodi già visitati, non può venire mandata al nodo iniziale se ci sono archi a nodi non visitati (anche fittizi). {deadlock, ANT, VISITED_AGENTS} ¡- messaggio in caso formica non possa muoversi su archi "normali"

1.3 Feature del sistema

loopACO: if userQueue =!= empty; msg = pop.userQueue(); dealWithUser(msg); for n in neighbourList n ! msg; if

1.4 Piano di testing?

1.5 Piano di implementazione?

Chapter 2

Analysis

In this chapter, we describe in detail functional and non-functional requirements of a solution for the problem.

2.1 Functional requirements

Which functions must be offered to users / other programs? Which are the input data and the output data? Which is the expected effect?

2.2 Non functional requirements

Everything about mode and transparencies: availability, mobility, security, fault tolerance, etc.

Are there execution time bounds? Minimum data rates?

If requested, specific platforms/languages/middlewares requirements for the implementation can be decided here. (E.g.: if the project is on a SOA, we may request that functions are offered via SOAP or RESTful services).

Chapter 3

Project

This chapter is devoted to the description of the general architectures, and specific algorithms.

3.1 Logical architecture

Describe the components of your systems: modules/objects/components/services. For each component, describe the functionalities it implements, and by who is used.

3.2 Protocols and algorithms

Communication between components. UML sequence diagrams go here.

Also, put here a detailed description of distributed algorithms used to solve specific problems of the project.

3.3 Physical architecture and deployment

Which nodes and platforms involved, and where each component is deployed.

3.4 Development plan

Since it is difficult to predict just how hard implementing a new system will be, you should formulate as a set of "tiers," where the basic tier is something you're sure you can complete, and the additional tiers add more features, at both the application and the system level.

Chapter 4

Implementation

Details about the implementation: every choice about platforms, languages, software/hardware, middlewares, which has not been decided in the requirements.

Important choices about implementation should be described here; e.g., peculiar data structures.

Chapter 5

Validation

Check if requirements from Chapter 2 have been fulfilled. Quantitative tests (simulations) and screenshots of the interfaces are put here.

Chapter 6

Conclusions

What has been done with respect to what has been promised in Chapters 1 and 2, and what is left out.

Appendix A

Appendix

In the Appendix you can put code snippets, snapshots, installation instructions, etc.

Evaluation

Your system will be judged mainly on how it operates as a distributed system. The primary evaluation will be according to whether your system has the following attributes:

- It should be an interesting distributed system, making use of some of the algorithms we have covered in class for distributed synchronization, replication, fault tolerance and recovery, security, etc.
- The software should be well designed and well implemented, in terms of the overall architecture and the detailed realization.
- You should devise and apply systematic testing procedures, at both the unit and systems levels.
- The system should operate reliably and with good performance, even in the face of failures.

Important, but secondary considerations include:

- Time taken to do the project (the sooner the better, but do not miss details in order to end sooner)
- How nice is the application's appearance: does it have a nice interface or a compelling visual display?